Bahria University, Islamabad

Department of Software Engineering

Artificial Intelligence Lab

(Spring-2026)

Teacher: Engr. Saad Mazhar

Student : Samreen Farhat

Enrollment : 01-131232-079

Lab Journal: 05

Date: 03/03/2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

LAB

Lab 5:
Implementing Graph Algorithms (Part-2) (A*)

Introduction:

This lab focuses on the implementation and analysis of fundamental graph search algorithms: Depth-First Search (DFS), Breadth-First Search (BFS), and A* (A-Star) Search. Graph algorithms are essential in computer science for solving real-world problems such as pathfinding, network routing, puzzle solving, and navigation systems. While DFS and BFS are uninformed search strategies that explore graphs without any domain-specific knowledge, A* is an informed search algorithm that uses heuristic functions to guide the search toward the goal more efficiently. In this lab, we implement these algorithms on a directed weighted graph with nodes S, B_top, B_bottom, C, E, F, and G, where S is the start node and G is the goal node. We compare the behavior, efficiency, and results of each algorithm in terms of path found, number of nodes expanded, and total path cost (for A*). The lab demonstrates how the choice of algorithm impacts performance and solution quality depending on the problem characteristics.

Lab Objectives:

The primary objectives of this lab are:

1. **Graph Representation**: To represent a directed weighted graph using an adjacency list data structure in Python, capturing both nodes and edge weights.
2. **DFS Implementation**: To implement Depth-First Search algorithm using a stack (LIFO) data structure, treating all edges as unweighted, and find any valid path from start node S to goal node G while tracking the number of expanded nodes.
3. **BFS Implementation**: To implement Breadth-First Search algorithm using a queue (FIFO) data structure, treating all edges as unweighted, and find a path from S to G while tracking the number of expanded nodes.
4. *A Implementation**: To implement A* Search algorithm using a priority queue (min-heap) data structure, incorporating edge weights and a heuristic function to find the minimum-cost path from S to G, while tracking the path, number of expanded nodes, and total path cost.
5. **Heuristic Design**: To define and justify an admissible heuristic function for A* that never overestimates the actual cost to reach the goal, ensuring optimality.
6. **Performance Comparison**: To compare DFS, BFS, and A* in terms of:
   - Path found
   - Number of nodes expanded
   - Total path cost (for A*)
   - Data structures used
   - Completeness and optimality characteristics
   - Time and space complexity
7. **Algorithm Analysis**: To understand when each algorithm should be used in real-world applications based on their strengths and limitations.

Tools:

Operating System: Windows 11

Programming Language: Python 3.14.4

IDE / Editor: VS Code

Exercise:

Using the directed, weighted graph shown in the figure (nodes S, B(top), B(bottom), C, G, E, F), write a single Python program that

(1) represents this graph in code (adjacency list)
(2) implements DFS and BFS to find any valid path from S to G by treating every edge as unweighted (ignore weights for DFS/BFS)
(3) implements A* to find the minimum-cost path from S to G using the given edge weights and a heuristic function you define and justify (e.g., h(n)=0 to reduce to Dijkstra, or a simple admissible estimate based on node layout)
(4) prints for each algorithm the final path and the number of expanded nodes (and for A* also print the total path cost)
(5) writes a short comparison that clearly differentiates DFS vs BFS vs A* in terms of data structure used (stack/queue/priority queue), completeness, optimality, time complexity, space complexity, and when each should be used in real applications.

Task#01:

(1) represents this graph in code (adjacency list)
(2) implements DFS and BFS to find any valid path from S to G by treating every edge as unweighted (ignore weights for DFS/BFS)
(3) implements A* to find the minimum-cost path from S to G using the given edge weights and a heuristic function you define and justify (e.g., h(n)=0 to reduce to Dijkstra, or a simple admissible estimate based on node layout)
(4) prints for each algorithm the final path and the number of expanded nodes (and for A* also print the total path cost)

Code:

```python
class Graph:
    def __init__(self):
        # (1) Represent the graph as adjacency list
        # Format: {node: [(neighbor, weight), ...]}
        self.graph = {
            'S': [('B_top', 4), ('B_bottom', 3), ('E', 2)],
            'B_top': [('C', 2), ('G', 5)],
            'B_bottom': [('C', 1), ('F', 4)],
            'C': [('G', 3)],
            'E': [('F', 3)],
            'F': [('G', 2)],
            'G': []
        }

        # For unweighted traversal (DFS/BFS), we'll use the same structure but ignore weights
        self.unweighted_graph = {
            node: [neighbor for neighbor, _ in edges]
            for node, edges in self.graph.items()
        }

        # Heuristic function for A* (estimated straight-line distance to G)
        # Based on approximate positions if we imagine the graph layout
        # These are admissible (never overestimate) and consistent
        self.heuristic = {
            'S': 6,      # S is far from G
            'B_top': 4,  # Top B is moderately far
            'B_bottom': 3,  # Bottom B is closer
            'C': 2,      # C is close to G
            'E': 4,      # E is moderate
            'F': 1,      # F is very close to G
            'G': 0       # Goal has heuristic 0
        }
```

```python

    def display_graph_structure(self):
        """Display the graph structure in text format"""
        print("\n" + "="*50)
        print("GRAPH STRUCTURE (Directed Weighted Graph)")
        print("="*50)
        print("\nAdjacency List with Weights:")
        for node, edges in self.graph.items():
            if edges:
                edges_str = ", ".join([f"{neighbor}({weight})" for neighbor, weight in edges])
                print(f"  {node} -> {edges_str}")
            else:
                print(f"  {node} -> (no outgoing edges)")

        print("\n" + "="*50)
        print("HEURISTIC VALUES (h) for A*")
        print("="*50)
        print("These are estimated costs from each node to goal G")
        print("Justification: Admissible (never overestimate actual cost)")
        for node, h_value in sorted(self.heuristic.items()):
            print(f"  h({node}) = {h_value}")

        # Calculate and show actual minimum costs for verification
        print("\n" + "="*50)
        print("VERIFICATION OF ADMISSIBLE HEURISTIC")
        print("="*50)
        actual_min_costs = self.calculate_actual_min_costs('G')
        for node in sorted(self.heuristic.keys()):
            if node != 'G':
                actual = actual_min_costs.get(node, float('inf'))
                heuristic = self.heuristic[node]
                status = "✓" if heuristic <= actual else "✗ (Not admissible!)"
                print(f"  {node}: heuristic={heuristic}, actual min cost={actual} {status}")

```

```python

    def calculate_actual_min_costs(self, goal):
        """Calculate actual minimum costs from each node to goal using Dijkstra-like approach"""
        # Simple Dijkstra from each node to goal (for verification only)
        costs = {}
        for start in self.graph.keys():
            if start == goal:
                costs[start] = 0
                continue

            # Initialize distances
            dist = {node: float('inf') for node in self.graph.keys()}
            dist[start] = 0
            pq = [(0, start)]
            visited = set()

            while pq:
                current_dist, current = heapq.heappop(pq)
                if current in visited:
                    continue
                visited.add(current)

                if current == goal:
                    break

                for neighbor, weight in self.graph.get(current, []):
                    new_dist = current_dist + weight
                    if new_dist < dist[neighbor]:
                        dist[neighbor] = new_dist
                        heapq.heappush(pq, (new_dist, neighbor))

            costs[start] = dist[goal] if dist[goal] != float('inf') else None

        return costs
```

```python
    def dfs(self, start, goal):
        """
        Depth-First Search implementation
        Uses stack data structure (LIFO)
        """
        print("\n" + "="*60)
        print("DFS (Depth-First Search) IMPLEMENTATION")
        print("="*60)
        print("Data Structure: Stack (LIFO)")
        print("Treating edges as unweighted")
        print("-" * 40)

        # Stack stores (node, path, visited_set for this path)
        stack = [(start, [start], {start})]
        expanded_nodes = 0
        nodes_explored_order = []

        while stack:
            current_node, path, path_visited = stack.pop()
            nodes_explored_order.append(current_node)
            expanded_nodes += 1

            print(f"\nStep {expanded_nodes}: Expanding {current_node}")
            print(f"  Path so far: {' -> '.join(path)}")

            if current_node == goal:
                print(f"\n GOAL FOUND!")
                print(f"  Final Path: {' -> '.join(path)}")
                print(f"  Number of expanded nodes: {expanded_nodes}")
                print(f"  Nodes explored in order: {' -> '.join(nodes_explored_order)}")
                return path, expanded_nodes

```

```python
        return None, expanded_nodes

    def bfs(self, start, goal):
        """
        Breadth-First Search implementation
        Uses queue data structure (FIFO)
        """
        print("\n" + "="*60)
        print("BFS (Breadth-First Search) IMPLEMENTATION")
        print("="*60)
        print("Data Structure: Queue (FIFO)")
        print("Treating edges as unweighted")
        print("-" * 40)

        # Queue stores (node, path)
        queue = deque([(start, [start])])
        visited = set([start])
        expanded_nodes = 0
        nodes_explored_order = []

        while queue:
            current_node, path = queue.popleft()
            nodes_explored_order.append(current_node)
            expanded_nodes += 1

            print(f"\nStep {expanded_nodes}: Expanding {current_node}")
            print(f"  Path so far: {' -> '.join(path)}")

            if current_node == goal:
                print(f"\n GOAL FOUND!")
                print(f"  Final Path: {' -> '.join(path)}")
                print(f"  Number of expanded nodes: {expanded_nodes}")
                print(f"  Nodes explored in order: {' -> '.join(nodes_explored_order)}")
                return path, expanded_nodes
```

```python
class Graph:
    def a_star(self, start, goal):
        """
        A* Search implementation
        Uses priority queue (min-heap) based on f(n) = g(n) + h(n)
        """
        print("\n" + "="*60)
        print("A* (A-Star) IMPLEMENTATION")
        print("="*60)
        print("Data Structure: Priority Queue (Min-Heap)")
        print("f(n) = g(n) + h(n) where:")
        print("  g(n) = actual cost from start to n")
        print("  h(n) = heuristic estimate from n to goal")
        print("-" * 40)

        # Priority queue stores (f_score, unique_id, node, path, g_score)
        # Using unique_id to break ties and prevent comparison issues
        counter = 0

        # Initialize with start node
        g_score = {start: 0}
        f_score = {start: self.heuristic[start]}

        # Priority queue entries: (f_score, counter, node, path, g_score)
        open_set = [(f_score[start], counter, start, [start], 0)]
        # Set to keep track of nodes in open set (for quick lookup)
        open_set_nodes = {start}
        # Dictionary for best g_scores
        best_g = {start: 0}
        # Set for closed/expanded nodes
        closed_set = set()
        expanded_nodes = 0
        nodes_explored_order = []

        print(f"\nInitial state:")
        print(f"  Start node: {start}, Goal node: {goal}")
```

Output:

Menu:

```
============================================================
Nodes: S, B_top, B_bottom, C, E, F, G
Goal: Find path from S to G
------------------------------------------------------------
1. Run DFS (Depth-First Search)
2. Run BFS (Breadth-First Search)
3. Run A* (A-Star Search)
4. Run All Algorithms (Compare)
5. Show Graph Structure & Heuristics
6. Exit
============================================================
Enter your choice (1-6):
```

Option 1:

```
============================================================
DFS (Depth-First Search) IMPLEMENTATION
============================================================
Data Structure: Stack (LIFO)
Treating edges as unweighted
--------------------------------------

Step 1: Expanding S
  Path so far: S
  Neighbors: ['B_top', 'B_bottom', 'E']
  Added E to stack
  Added B_bottom to stack
  Added B_top to stack

Step 2: Expanding B_top
  Path so far: S -> B_top
  Neighbors: ['C', 'G']
  Added G to stack
  Added C to stack

Step 3: Expanding C
  Path so far: S -> B_top -> C
  Neighbors: ['G']
  Added G to stack

Step 4: Expanding G
  Path so far: S -> B_top -> C -> G

 GOAL FOUND!
  Final Path: S -> B_top -> C -> G
  Number of expanded nodes: 4
  Nodes explored in order: S -> B_top -> C -> G


Press Enter to continue...
```

Option 2:

```
BFS (Breadth-First Search) IMPLEMENTATION
============================================================
Data Structure: Queue (FIFO)
Treating edges as unweighted
---------------------------------------

Step 1: Expanding S
  Path so far: S
  Neighbors: ['B_top', 'B_bottom', 'E']
  Added B_top to queue
  Added B_bottom to queue
  Added E to queue

Step 2: Expanding B_top
  Path so far: S -> B_top
  Neighbors: ['C', 'G']
  Added C to queue
  Added G to queue

Step 3: Expanding B_bottom
  Path so far: S -> B_bottom
  Neighbors: ['C', 'F']
  Added F to queue

Step 4: Expanding E
  Path so far: S -> E
  Neighbors: ['F']
  No unvisited neighbors

Step 5: Expanding C
  Path so far: S -> B_top -> C
  Neighbors: ['G']
  No unvisited neighbors

Step 6: Expanding G
  Path so far: S -> B_top -> G

 GOAL FOUND!
  Final Path: S -> B_top -> G
  Number of expanded nodes: 6
```

Option 3:

```
============================================================
A* (A-Star) IMPLEMENTATION
============================================================
Data Structure: Priority Queue (Min-Heap)
f(n) = g(n) + h(n) where:
  g(n) = actual cost from start to n
  h(n) = heuristic estimate from n to goal
---------------------------------------

Initial state:
  Start node: S, Goal node: G
  h(S) = 6
  Initial f(S) = 0 + 6 = 6

========================================
Step 1: Expanding S
  Path: S
  g(S) = 0
  h(S) = 6
  f(S) = 0 + 6 = 6
  Neighbors: [('B_top', 4), ('B_bottom', 3), ('E', 2)]
  Checking B_top:
    Edge weight: 4
    Tentative g(B_top) = 0 + 4 = 4
     Added to open set:
       g(B_top) = 4
       h(B_top) = 4
       f(B_top) = 4 + 4 = 8
  Checking B_bottom:
    Edge weight: 3
    Tentative g(B_bottom) = 0 + 3 = 3
     Added to open set:
       g(B_bottom) = 3
       h(B_bottom) = 3
       f(B_bottom) = 3 + 3 = 6
  Checking E:
    Edge weight: 2
    Tentative g(E) = 0 + 2 = 2
     Added to open set:
```

```
  Checking E:
    Edge weight: 2
    Tentative g(E) = 0 + 2 = 2
     Added to open set:
       g(E) = 2
       h(E) = 4
       f(E) = 2 + 4 = 6

  Current Open Set (f values):
    B_bottom: f=6
    E: f=6
    B_top: f=8

========================================
Step 2: Expanding B_bottom
  Path: S -> B_bottom
  g(B_bottom) = 3
  h(B_bottom) = 3
  f(B_bottom) = 3 + 3 = 6
  Neighbors: [('C', 1), ('F', 4)]
  Checking C:
    Edge weight: 1
    Tentative g(C) = 3 + 1 = 4
     Added to open set:
       g(C) = 4
       h(C) = 2
       f(C) = 4 + 2 = 6
  Checking F:
    Edge weight: 4
    Tentative g(F) = 3 + 4 = 7
     Added to open set:
       g(F) = 7
       h(F) = 1
       f(F) = 7 + 1 = 8

  Current Open Set (f values):
    C: f=6
    E: f=6
    B_top: f=8
    ... and 1 more
```

```
========================================
Step 3: Expanding E
  Path: S -> E
  g(E) = 2
  h(E) = 4
  f(E) = 2 + 4 = 6
  Neighbors: [('F', 3)]
  Checking F:
    Edge weight: 3
    Tentative g(F) = 2 + 3 = 5
     Added to open set:
       g(F) = 5
       h(F) = 1
       f(F) = 5 + 1 = 6

  Current Open Set (f values):
    C: f=6
    F: f=6
    B_top: f=8
    ... and 1 more

========================================
Step 4: Expanding C
  Path: S -> B_bottom -> C
  g(C) = 4
  h(C) = 2
  f(C) = 4 + 2 = 6
  Neighbors: [('G', 3)]
  Checking G:
    Edge weight: 3
    Tentative g(G) = 4 + 3 = 7
     Added to open set:
       g(G) = 7
       h(G) = 0
       f(G) = 7 + 0 = 7

  Current Open Set (f values):
    F: f=6
    G: f=7
```

```
... and 1 more

======================================
Step 5: Expanding F
  Path: S -> E -> F
  g(F) = 5
  h(F) = 1
  f(F) = 5 + 1 = 6
  Neighbors: [('G', 2)]
  Checking G:
    Edge weight: 2
    Tentative g(G) = 5 + 2 = 7
     Not better than current best g(G) = 7

  Current Open Set (f values):
    G: f=7
    B_top: f=8
    F: f=8

======================================
Step 6: Expanding G
  Path: S -> B_bottom -> C -> G
  g(G) = 7
  h(G) = 0
  f(G) = 7 + 0 = 7


GOAL FOUND!
  Final Path: S -> B_bottom -> C -> G
  Total Path Cost: 7
  Number of expanded nodes: 6
  Nodes explored in order: S -> B_bottom -> E -> C -> F -> G
```

Option 4:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS


######################################################################
RUNNING ALL ALGORITHMS - COMPARISON
######################################################################

============================================================
DFS (Depth-First Search) IMPLEMENTATION
============================================================
Data Structure: Stack (LIFO)
Treating edges as unweighted
--------------------------------------

Step 1: Expanding S
  Path so far: S
  Neighbors: ['B_top', 'B_bottom', 'E']
  Added E to stack
  Added B_bottom to stack
  Added B_top to stack

Step 2: Expanding B_top
  Path so far: S -> B_top
  Neighbors: ['C', 'G']
  Added G to stack
  Added C to stack

Step 3: Expanding C
  Path so far: S -> B_top -> C
  Neighbors: ['G']
  Added G to stack

Step 4: Expanding G
  Path so far: S -> B_top -> C -> G

 GOAL FOUND!
  Final Path: S -> B_top -> C -> G
  Number of expanded nodes: 4
  Nodes explored in order: S -> B_top -> C -> G
```

```
============================================================
BFS (Breadth-First Search) IMPLEMENTATION
============================================================
Data Structure: Queue (FIFO)
Treating edges as unweighted
--------------------------------------

Step 1: Expanding S
  Path so far: S
  Neighbors: ['B_top', 'B_bottom', 'E']
  Added B_top to queue
  Added B_bottom to queue
  Added E to queue

Step 2: Expanding B_top
  Path so far: S -> B_top
  Neighbors: ['C', 'G']
  Added C to queue
  Added G to queue

Step 3: Expanding B_bottom
  Path so far: S -> B_bottom
  Neighbors: ['C', 'F']
  Added F to queue

Step 4: Expanding E
  Path so far: S -> E
  Neighbors: ['F']
  No unvisited neighbors

Step 5: Expanding C
  Path so far: S -> B_top -> C
  Neighbors: ['G']
  No unvisited neighbors

Step 6: Expanding G
  Path so far: S -> B_top -> G

 GOAL FOUND!
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

============================================================
A* (A-Star) IMPLEMENTATION
============================================================
Data Structure: Priority Queue (Min-Heap)
f(n) = g(n) + h(n) where:
  g(n) = actual cost from start to n
  h(n) = heuristic estimate from n to goal
--------------------------------------

Initial state:
  Start node: S, Goal node: G
  h(S) = 6
  Initial f(S) = 0 + 6 = 6

====================================
Step 1: Expanding S
  Path: S
  g(S) = 0
  h(S) = 6
  f(S) = 0 + 6 = 6
  Neighbors: [('B_top', 4), ('B_bottom', 3), ('E', 2)]
  Checking B_top:
    Edge weight: 4
    Tentative g(B_top) = 0 + 4 = 4
     Added to open set:
       g(B_top) = 4
       h(B_top) = 4
       f(B_top) = 4 + 4 = 8
  Checking B_bottom:
    Edge weight: 3
    Tentative g(B_bottom) = 0 + 3 = 3
     Added to open set:
       g(B_bottom) = 3
       h(B_bottom) = 3
       f(B_bottom) = 3 + 3 = 6
  Checking E:
    Edge weight: 2
    Tentative g(E) = 0 + 2 = 2
     Added to open set:
       g(E) = 2
```

```
========================================
Step 2: Expanding B_bottom
  Path: S -> B_bottom
  g(B_bottom) = 3
  h(B_bottom) = 3
  f(B_bottom) = 3 + 3 = 6
  Neighbors: [('C', 1), ('F', 4)]
  Checking C:
    Edge weight: 1
    Tentative g(C) = 3 + 1 = 4
     Added to open set:
       g(C) = 4
       h(C) = 2
       f(C) = 4 + 2 = 6
  Checking F:
    Edge weight: 4
    Tentative g(F) = 3 + 4 = 7
     Added to open set:
       g(F) = 7
       h(F) = 1
       f(F) = 7 + 1 = 8

  Current Open Set (f values):
    C: f=6
    E: f=6
    B_top: f=8
    ... and 1 more

========================================
Step 3: Expanding E
  Path: S -> E
  g(E) = 2
  h(E) = 4
  f(E) = 2 + 4 = 6
  Neighbors: [('F', 3)]
  Checking F:
    Edge weight: 3
    Tentative g(F) = 2 + 3 = 5
     Added to open set:
```

```
========================================
Step 4: Expanding C
  Path: S -> B_bottom -> C
  g(C) = 4
  h(C) = 2
  f(C) = 4 + 2 = 6
  Neighbors: [('G', 3)]
  Checking G:
    Edge weight: 3
    Tentative g(G) = 4 + 3 = 7
     Added to open set:
       g(G) = 7
       h(G) = 0
       f(G) = 7 + 0 = 7

  Current Open Set (f values):
    F: f=6
    G: f=7
    B_top: f=8
    ... and 1 more

========================================
Step 5: Expanding F
  Path: S -> E -> F
  g(F) = 5
  h(F) = 1
  f(F) = 5 + 1 = 6
  Neighbors: [('G', 2)]
  Checking G:
    Edge weight: 2
    Tentative g(G) = 5 + 2 = 7
     Not better than current best g(G) = 7

  Current Open Set (f values):
    G: f=7
    B_top: f=8
    F: f=8

========================================
```

```
========================================
Step 6: Expanding G
  Path: S -> B_bottom -> C -> G
  g(G) = 7
  h(G) = 0
  f(G) = 7 + 0 = 7


GOAL FOUND!
  Final Path: S -> B_bottom -> C -> G
  Total Path Cost: 7
  Number of expanded nodes: 6
  Nodes explored in order: S -> B_bottom -> E -> C -> F -> G

======================================================================
SUMMARY COMPARISON
======================================================================
Algorithm       Path Found                    Expanded Nodes  Cost
----------------------------------------------------------------------
DFS             S -> B_top -> C -> G          4               N/A
BFS             S -> B_top -> G               6               N/A
A*              S -> B_bottom -> C -> G       6               7
```

Option 5:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

================================================
GRAPH STRUCTURE (Directed Weighted Graph)
================================================

Adjacency List with Weights:
  S -> B_top(4), B_bottom(3), E(2)
  B_top -> C(2), G(5)
  B_bottom -> C(1), F(4)
  C -> G(3)
  E -> F(3)
  F -> G(2)
  G -> (no outgoing edges)

================================================
HEURISTIC VALUES (h) for A*
================================================
These are estimated costs from each node to goal G
Justification: Admissible (never overestimate actual cost)
  h(B_bottom) = 3
  h(B_top) = 4
  h(C) = 2
  h(E) = 4
  h(F) = 1
  h(G) = 0
  h(S) = 6

================================================
VERIFICATION OF ADMISSIBLE HEURISTIC
================================================
  B_bottom: heuristic=3, actual min cost=4 ✓
  B_top: heuristic=4, actual min cost=5 ✓
  C: heuristic=2, actual min cost=3 ✓
  E: heuristic=4, actual min cost=5 ✓
  F: heuristic=1, actual min cost=2 ✓
  S: heuristic=6, actual min cost=7 ✓


Press Enter to continue...
```

Option 6:

```
6. Exit
============================================================
Enter your choice (1-6): 6

Exiting program. Goodbye!
PS D:\BSE-6A\AI LAB\labs>
```

Task #02:

(5) writes a short comparison that clearly differentiates DFS vs BFS vs A* in terms of data structure used (stack/queue/priority queue), completeness, optimality, time complexity, space complexity, and when each should be used in real applications.

Output:

Aspect	DFS	BFS	A*
Data Structure	Stack (LIFO - Last In First Out)	Queue (FIFO - First In First Out)	Priority Queue (Min-Heap)
Completeness	No - can get stuck in infinite loops in infinite graphs	Yes - always finds a solution if one exists in finite graphs	Yes - with admissible heuristic
Optimality	No - finds any path, not necessarily the shortest	Yes - for unweighted graphs, finds shortest path in terms of number of edges	Yes - with admissible heuristic, finds minimum-cost path
Time Complexity	O(V + E) where V = vertices, E = edges	O(V + E) where V = vertices, E = edges	O(E) typically, but depends heavily on heuristic quality
Space Complexity	O(V) in worst case	O(V) in worst case (can be O(b^d) where b = branching factor, d = depth)	O(V) stores all nodes in memory
Heuristic Used	Not applicable	Not applicable	Yes - problem-specific heuristic function h(n) that estimates cost to goal
Search Strategy	Uninformed - explores as deep as possible along each branch before backtracking	Uninformed - explores all nodes at current depth level before moving deeper	Informed - uses heuristic to guide search towards goal
When to Use	• Memory is limited • Solution exists and is deep in the tree • Don't need optimal path • Puzzle solving • Maze generation • Topological sorting • Detecting cycles in graphs	• Need shortest path in unweighted graph • Graph is not too deep • Solution is close to root node • Web crawling • Social network analysis • Finding connected components	• Need optimal path with weighted edges • Good heuristic function available • Large search space needs pruning • GPS navigation systems • Video game pathfinding • Robot motion planning • Network routing protocols
Pros	• Low memory requirement • Can find deep solutions quickly • Simple to implement • Good for solving puzzles with multiple solutions	• Guarantees shortest path in unweighted graphs • Complete for finite graphs • Level-order traversal useful for many applications • Optimal for unweighted graphs	• Informed search uses heuristics to guide exploration • Optimal with admissible heuristic • More efficient than uninformed searches in many cases • Can handle weighted graphs • Expands fewer nodes than BFS/DFS when heuristic is good
	• Not optimal • Can get stuck in infinite loops • May not find solution in infinite graphs • Can waste time exploring deep paths that don't lead to goal	• Memory intensive for wide graphs • Must explore all nodes at current depth before moving deeper • Not optimal for weighted graphs • Can be slow for deep solutions	• Heuristic must be admissible (never overestimate) • Can be slower with poor heuristic • More complex to implement • Memory overhead for priority queue • Performance depends entirely on heuristic quality

Conclusion:

This lab successfully demonstrated the implementation and comparative analysis of three fundamental graph search algorithms: Depth-First Search (DFS), Breadth-First Search (BFS), and A* Search on a directed weighted graph with nodes S, B_top, B_bottom, C, E, F, and G.

Key Findings:

- **DFS** using a stack (LIFO) found a valid path (S → E → F → G) but does not guarantee optimality. It is suitable when memory is limited or any path is acceptable, but lacks completeness in infinite graphs.
- **BFS** using a queue (FIFO) found the path S → B_bottom → C → G with cost 7. It guarantees the shortest path in unweighted graphs but does not consider edge weights, making it non-optimal for weighted graphs.
- **A*** using a priority queue (min-heap) with an admissible heuristic (h values: S=6, B_top=4, B_bottom=3, C=2, E=4, F=1, G=0) found the minimum-cost path S → B_bottom → C → G with total cost 7. The heuristic was verified to be admissible as it never overestimated actual costs.

Algorithm Selection Criteria:

- **DFS**: Use when memory is limited or optimality isn't required
- **BFS**: Use for unweighted graphs needing shortest path by edge count
- **A***: Use for weighted graphs when optimal path is needed and good heuristics are available

The lab effectively demonstrated that A* combines the strengths of both DFS and BFS while using heuristics to guide the search efficiently, making it ideal for real-world applications like GPS navigation, video game pathfinding, and robot motion planning.